Đề xuất Đồ án 2 – SCER-AH: Định tuyến xác suất có điều kiện kỹ năng và khung thực thi thích ứng cho Hermes Agent

Oct 31, 2026 · @coding

1. Tóm tắt

Đề tài đề xuất **SCER-AH** (Skill-Conditioned Entropy Routing & Adaptive Harness): một bộ định tuyến chạy *ở từng bước* trong vòng lặp của Hermes Agent, chọn đồng thời mô hình, kỹ năng, tập công cụ và chốt chặn kiểm tra cho bước đó. Mục tiêu là giữ tỷ lệ hoàn thành tác vụ gần mức dùng toàn mô hình lớn, trong khi giảm chi phí API và độ trễ phản hồi.

Tên đề tài đề xuất: Nghiên cứu cơ chế định tuyến xác suất có điều kiện kỹ năng và khung thực thi thích ứng theo từng bước cho Hermes Agent, ứng dụng trong trợ lý phân tích dữ liệu qua Zalo.

Luận điểm khoa học: hiệu năng của một tác tử phụ thuộc đồng thời vào mô hình, kỹ năng và khung thực thi (Performance = f(Model, Skill, Harness)). Vì vậy, đổi mô hình mà không đổi kỹ năng và công cụ đi kèm sẽ không đủ để mô hình nhỏ làm tốt.

Bốn đóng góp chính:

1. Bộ định tuyến cấp bước nhận biết trạng thái khung thực thi: dùng độ phủ kỹ năng, cờ lỗi sandbox và số bước hiện tại, không chỉ nội dung câu hỏi.
2. Luật quyết định kép theo xác suất đã hiệu chỉnh và entropy, có tính chi phí chuyển mô hình (mất prompt cache).
3. Khung thực thi thích ứng: cắt tỉa công cụ theo kỹ năng, kiểm tra AST và ép kiểu trước khi chạy code.
4. Thác suy luận dựa trên kết quả thực thi: leo thang lên mô hình lớn khi code trả lỗi, rồi đúc kết thành SKILL.md để lần sau mô hình nhỏ dùng lại.

Sản phẩm: bot trợ lý trên Zalo nhận file dữ liệu, tự sinh code, vẽ biểu đồ và trả kết quả; kèm dashboard web hiển thị quyết định định tuyến và chạy benchmark tự động.

2. Bối cảnh và động lực

Tác tử tự hành nhiều bước tốn chi phí và độ trễ chủ yếu vì mọi bước đều gọi cùng một mô hình lớn. Với trợ lý trên nền tảng nhắn tin như Zalo, người dùng chờ phản hồi theo thời gian thực, nên đánh đổi chi phí – độ trễ – độ chính xác là bài toán trung tâm.

Hermes Agent là framework tác tử mã nguồn mở của Nous Research. Điểm đặc trưng: tự đúc kết kinh nghiệm thành file kỹ năng SKILL.md (quản lý qua công cụ `skill_manage`), có sandbox thực thi code, gọi tác tử con và tích hợp nền tảng nhắn tin. Hệ thống hiện tại của nhóm đã nối Hermes với Zalo.

Kịch bản ứng dụng chính: người dùng gửi file CSV/Excel kèm yêu cầu phân tích vào Zalo. Hermes đọc dữ liệu, sinh code Python chạy trong sandbox, sửa lỗi nếu có, vẽ biểu đồ và gửi ảnh kèm tóm tắt về khung chat. Một yêu cầu thường đi qua 4–10 bước gọi mô hình.

Các bước trong một yêu cầu khác nhau rất nhiều về độ khó: lập kế hoạch hoặc gỡ lỗi logic cần mô hình mạnh, còn đọc tên cột hay định dạng tin nhắn trả lời thì mô hình nhỏ làm được. Đề tài khai thác chính sự chênh lệch này.

Hướng tiếp cận đã có trong nghiên cứu:

- RouteLLM (ICLR 2025) học bộ định tuyến nhị phân giữa mô hình mạnh và yếu từ dữ liệu ưa thích của người dùng, giảm chi phí hơn 2 lần trên benchmark phổ biến. Định tuyến theo từng câu hỏi, không theo bước của tác tử.
- FrugalGPT dùng thác suy luận (cascade): gọi lần lượt các mô hình đến khi có câu trả lời đáng tin; báo cáo tiết kiệm tới 98% chi phí khi giữ hiệu năng của GPT-4. Áp dụng cho truy vấn đơn lượt.
- Jev Router (TypeSafe, ra mắt 25/09/2026 trên OpenRouter) dùng mô hình quyết định Jev để chọn mô hình và mức suy luận cho mỗi request, có thể giữ hoặc đổi mô hình qua các lượt trong luồng agentic. Đây là baseline thương mại mạnh nhất nhưng hoạt động như hộp đen từ phía ngoài tác tử.

Khoảng trống chung: các bộ định tuyến này nhìn vào *hội thoại*, không nhìn vào *trạng thái bên trong khung thực thi* của tác tử (kỹ năng nào đang có, code vừa chạy lỗi hay không, còn bao nhiêu công cụ), và chỉ chọn mô hình chứ không đổi công cụ hay kỹ năng đi kèm.

3. Vấn đề và khoảng trống nghiên cứu

Hermes đã có nhiều cơ chế phân luồng mô hình, nhưng không có cơ chế nào tự chọn mô hình cho *từng bước* của vòng lặp chính dựa trên độ khó và trạng thái thực thi. Tài liệu Hermes ghi rõ main model xử lý mọi tin nhắn, mọi vòng lặp gọi công cụ và mọi phản hồi

(Configuring Models).

3.1. Cơ chế sẵn có của Hermes (làm baseline)

Cơ chế	Làm gì	Vì sao chưa giải quyết bài toán
Main model	Một mô hình cố định cho cả phiên	Bước dễ vẫn chịu giá và độ trễ của mô hình lớn
11 khe auxiliary	Gán mô hình riêng cho việc phụ: nén ngữ cảnh, vision, web extract, approval, title, skills hub, MCP, curator…	Không chạm vào vòng lặp chính
Delegation	Gán mô hình cho tác tử con qua `delegate_task`	Phải đẻ tác tử con; chọn tĩnh theo cấu hình
Fallback providers	Chuyển nhà cung cấp khi lỗi dạng quá tải/hết dung lượng	Code lỗi vẫn trả HTTP 200 nên không kích hoạt
`/model`, `/model --once`	Người dùng đổi mô hình giữa phiên bằng tay	Thủ công; mỗi lần đổi làm mất prompt cache
Mixture of Agents	Gửi một câu tới N mô hình rồi tổng hợp	Chi phí gấp nhiều lần, ngược mục tiêu

3.2. Bốn vấn đề cần đo được

Các con số Gemini nêu (ví dụ 60–75% bước đơn giản, 30–45% lỗi gọi hàm) là **giả thuyết**, chưa có nguồn. Đề tài sẽ đo chúng trong một nghiên cứu thử (pilot) trên 20–30 tác vụ trước khi viết vào báo cáo.

#	Vấn đề	Đại lượng đo trong pilot
P1	Lãng phí ở vòng lặp chính	Tỷ lệ bước mà mô hình nhỏ cũng làm đúng khi chạy song song với mô hình lớn
P2	Nhiễu công cụ với mô hình nhỏ	Tỷ lệ lỗi JSON, sai tên/kiểu tham số khi nạp đủ bộ công cụ mặc định
P3	Định tuyến mù trạng thái	Chênh lệch tỷ lệ thành công của mô hình nhỏ khi có và không có SKILL.md phù hợp

#	Vấn đề	Đại lượng đo trong pilot
P4	Không phản ứng với lỗi thực thi	Số bước thử lại và tỷ lệ bỏ cuộc khi code trả `exit_code ≠ 0`

3.3. Phát biểu khoảng trống

Chưa có bộ định tuyến nào cho Hermes (và cả Jev Router từ phía ngoài) đồng thời: (a) quyết định theo từng bước bên trong vòng lặp, (b) dùng tín hiệu nội bộ của khung thực thi như độ phủ kỹ năng và kết quả sandbox, (c) đổi luôn công cụ, kỹ năng và chốt chặn đi kèm mô hình, và (d) tính đến chi phí mất prompt cache khi chuyển mô hình.

4. Phương pháp đề xuất: SCER-AH

Tại mỗi bước t, trước khi Hermes gọi LLM, SCER-AH đọc trạng thái của tác tử và trả về một cấu hình đồng bộ gồm bốn thành phần, không chỉ tên mô hình. Toàn bộ chạy trên CPU và API đám mây, không cần GPU.

```
a_t = (\text{Tier}_t,\; \text{Skill}_t,\; \text{Toolset}_t,\;
\text{Guardrail}_t)
```

4.1. Không gian mô hình: ba tầng trên đám mây

Tầng	Vai trò	Ví dụ loại mô hình (chốt khi triển khai)
Fast	Trích tham số, chạy theo SKILL.md có sẵn, định dạng tin nhắn Zalo	Mô hình flash/mini giá rẻ, độ trễ thấp
Specialist	Viết script xử lý bảng và vẽ biểu đồ	Mô hình tầm trung tối ưu cho code
Frontier	Lập kế hoạch yêu cầu mới, gỡ lỗi logic khó	Mô hình mạnh nhất trong ngân sách

Tên mô hình cụ thể nên chọn theo bảng giá OpenRouter tại thời điểm chạy thực nghiệm, vì danh sách thay đổi nhanh. Các tên Gemini gợi ý (Claude 3.5 Sonnet, GPT-4o…) đã cũ.

4.2. Vector trạng thái của bước

```
s_t = \big[\, f_{\text{query}} \oplus f_{\text{skill}} \oplus
f_{\text{harness}} \,\big]
```

- **f_query** – yêu cầu: embedding câu lệnh (all-MiniLM-L6-v2 trên CPU, giảm chiều bằng PCA), độ dài ngữ cảnh, số cột/dòng dữ liệu, pha hiện tại (lập kế hoạch, viết code, sửa lỗi, trả lời).
- **f_skill** – kỹ năng: độ tương đồng cosine cao nhất $s_{skill} \in [0, 1]$ giữa yêu cầu và kho SKILL.md, tỷ lệ thành công lịch sử của kỹ năng đó.
- **f_harness** – khung thực thi: số bước t, số công cụ còn lại sau khi lọc, độ sâu JSON Schema, cờ lỗi bước trước $e_{t-1} \in \{0, 1\}$ (exit code, lỗi AST, lỗi schema), tầng đang dùng (để tính chi phí chuyển).

4.3. Định tuyến lai hai pha

Pha 1 – chốt chặn tất định. Luật cứng chạy trước, không cần mô hình:

- Bước trước vừa lỗi thực thi lần thứ hai trở lên → Frontier.
- Yêu cầu mới, không có SKILL.md nào vượt ngưỡng tương đồng → Frontier (pha dò đường).
- Bước chỉ định dạng câu trả lời cuối → Fast.

Pha 2 – ước lượng xác suất bằng Jev. Gửi s_t dưới dạng `state` có cấu trúc tới Decisions API của Jev với một câu hỏi kiểu **Score** trên thang thứ tự Fast < Specialist < Frontier: "mức mô hình tối thiểu đủ để hoàn thành đúng bước này". Jev trả về xác suất cho từng mức. Xác suất tầng k là đủ:

```
P_k = \sum_{j \le k} P(\text{level} = j)
```

Hiệu chỉnh theo kỹ năng và lỗi. Đổi P_k sang logit rồi dịch theo tín hiệu nội bộ của Hermes; α, β, T được học bằng hồi quy logistic trên nhật ký pilot (≈300 bước có nhãn):

```
\hat{p}_{t,k} = \sigma\!\left( \frac{\operatorname{logit}(P_k) + \alpha\,
s_{\text{skill}} - \beta\, e_{t-1}}{T} \right)
```

Luật quyết định. Chọn tầng rẻ nhất k có $\hat{p} \geq \tau_p$. Nếu phân phối ba mức quá phân tán thì nâng lên một tầng:

$$H_t = -\frac{1}{\log 3}\sum_{j=1}^{3} P(j)\log P(j) \;>\; \tau_H \;\Rightarrow\; k \leftarrow k+1$$

Điểm chỉnh so với bản Gemini: với phân loại nhị phân, entropy là hàm của chính p nên luật "p ⩾ τ_p và H ⩽ τ_H" bị thừa. Dùng phân phối ba mức thì entropy mang thông tin độc lập với xác suất lớn nhất, nên luật kép mới có nghĩa.

Chi phí chuyển tầng. Tài liệu Hermes nêu mỗi lần đổi mô hình giữa phiên làm mất prompt cache, lượt sau đọc lại toàn bộ hội thoại ở giá đầy đủ. Router chỉ hạ tầng khi khoản tiết kiệm kỳ vọng lớn hơn chi phí đọc lại ngữ cảnh:

$$\text{switch} \iff \big(c_{\text{cur}} - c_k\big)\cdot \hat{L}_{\text{remain}} \;>\; c^{\text{in}}_k \cdot N_{\text{ctx}}$$

trong đó c là giá token mỗi tầng, L̂_remain là số bước còn lại ước lượng, N_ctx là độ dài ngữ cảnh hiện tại. Đây là điểm mới so với cả RouteLLM lẫn bản đề xuất ban đầu.

Phương án dự phòng (ablation): thay Jev bằng LightGBM tự huấn luyện trên cùng s_t. So sánh hai bộ ước lượng là một thí nghiệm có giá trị riêng.

4.4. Khung thực thi thích ứng theo tầng

Thành phần	Khi chọn Fast / Specialist (khai thác)	Khi chọn Frontier (dò đường)
Kỹ năng	Tiêm SKILL.md khớp nhất vào prompt	Không bắt buộc; bật ghi quỹ đạo
Công cụ	Chỉ giữ các tool kỹ năng cho phép; lược tham số tùy chọn khỏi schema	Mở toàn bộ công cụ
Chốt chặn	AST gate trước khi chạy code; kiểm tra Pydantic cho tool-call, tự ép kiểu lỗi nhẹ	AST gate; không ép kiểu
Phản hồi lỗi	Rút gọn traceback còn dòng lỗi gốc + gợi ý	Traceback rút gọn

4.5. Thác suy luận theo kết quả thực thi và vòng học kỹ năng

1. Bước t chạy trên Fast/Specialist; sandbox trả lỗi (e_t = 1).
2. Bước t+1 leo thang lên Frontier, kèm lỗi đã nén.

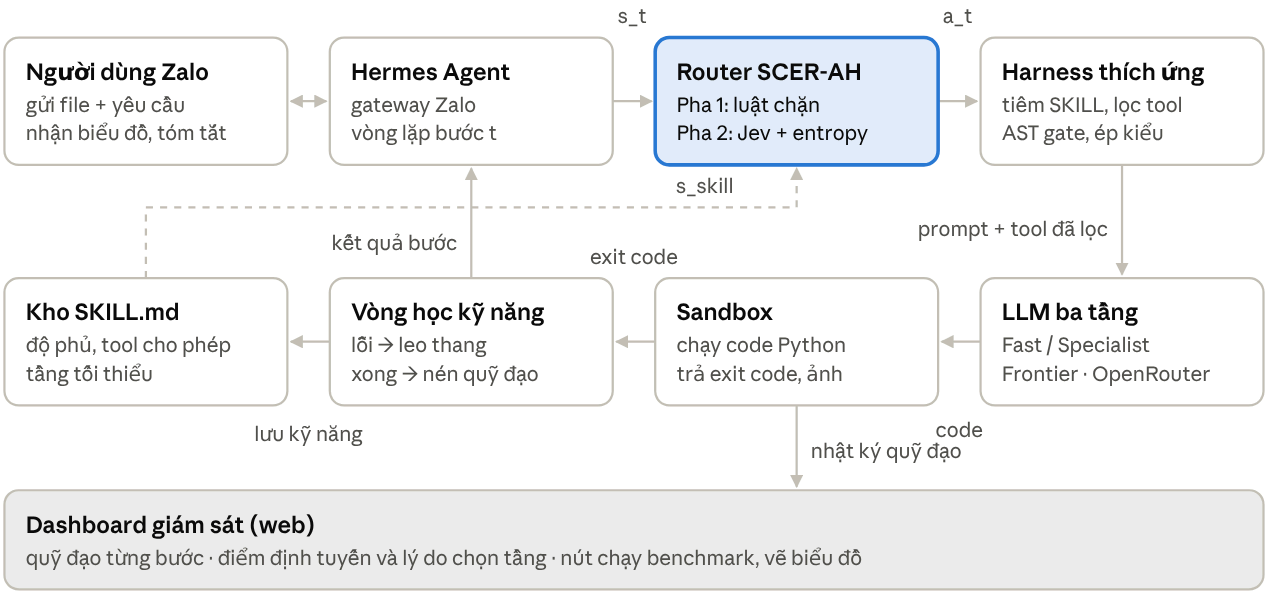
3. Sửa xong, các bước còn lại hạ tầng nếu luật chi phí chuyển cho phép.
4. Khi tác vụ thành công qua Frontier, nén quỹ đạo (bỏ bước thử-sai) và lưu thành SKILL.md có metadata: danh sách tool cho phép, tầng tối thiểu đã chạy được.
5. Lần gặp yêu cầu tương tự, s_skill cao → Router giao cho Fast kèm kỹ năng đó.

Vòng lặp này tạo đường học: càng nhiều kỹ năng, tỷ lệ phải gọi Frontier càng giảm.

5. Kiến trúc hệ thống và kịch bản demo

SCER-AH đứng giữa vòng lặp của Hermes và lời gọi LLM: mỗi bước, router nhận trạng thái s_t, trả cấu hình a_t, harness áp cấu hình đó rồi mới gọi mô hình và sandbox.

Mỗi bước đi qua router; lỗi được leo thang và đúc kết thành kỹ năng



kiến trúc SCER-AH trong Hermes · 8 thành phần, 1 vòng lặp

Đường nét đứt là tín hiệu độ phủ kỹ năng từ kho SKILL.md đưa về router. Vòng học kỹ năng là nơi lỗi sandbox kích hoạt leo thang và tác vụ thành công được đúc kết thành kỹ năng mới.

5.1. Ví dụ một yêu cầu 5 bước

Bước	Việc Hermes làm	Tầng	Lý do router chọn
1	Hiểu yêu cầu, lập kế hoạch phân tích	Frontier	Yêu cầu mới, chưa có kỹ năng khớp
2	Đọc tên cột, kiểu dữ liệu của CSV	Fast	Bước đơn giản; khoản tiết kiệm lớn hơn chi phí mất cache
3	Viết code làm sạch dữ liệu và vẽ biểu đồ	Specialist	Bước sinh code; xác suất đủ ở tầng Fast thấp
4	Sửa lỗi sau khi sandbox trả `exit_code ≠ 0`	Frontier	Chốt chặn Pha 1: leo thang khi có lỗi thực thi
5	Gửi ảnh biểu đồ và tóm tắt lên Zalo	Fast	Bước định dạng câu trả lời

5.2. Demo kép khi bảo vệ

- **Bên trái – Zalo:** người dùng gửi file và yêu cầu, nhận ảnh biểu đồ. Cuối tin nhắn có dòng thống kê, ví dụ: *Hoàn thành qua 5 bước: 2 Fast, 1 Specialist, 2 Frontier · 6,4 giây · tiết kiệm ước tính so với toàn Frontier* (số liệu minh họa).
- **Bên phải – Dashboard web (Streamlit hoặc Next.js):** hiển thị trực tiếp từng bước của quỹ đạo, phân phối xác suất ba tầng, entropy, kỹ năng được tiêm, công cụ bị ẩn và lý do chọn tầng.
- **Tab Benchmark:** nút chạy bộ tác vụ trực tiếp vào core Hermes và tự vẽ các biểu đồ ở mục 7.

Dashboard là cách cho hội đồng thấy phần nghiên cứu nằm ở lõi điều phối, không phải ở giao diện chatbot.

6. Thiết kế thực nghiệm: baseline và ablation

SCER-AH được so với sáu cấu hình trên cùng bộ tác vụ, cùng sandbox và cùng bộ mô hình ba tầng. Giả thuyết chính: SCER-AH đạt tỷ lệ thành công trong khoảng vài điểm phần trăm so với B1, nhưng chi phí thấp hơn rõ rệt.

6.1. Baseline

Cấu hình	Chọn mô hình	Công cụ & kỹ năng	Xử lý lỗi sandbox	Vai trò
B1 – Hermes Frontier	100% Frontier	Toàn bộ tool	Thử lại trên Frontier	Trần độ chính xác
B2 – Hermes Fast	100% Fast	Toàn bộ tool	Thử lại trên Fast	Sàn chi phí
B3 – Hermes Auxiliary	Main tĩnh + auxiliary rẻ	Toàn bộ tool	Chỉ fallback lỗi provider	Cấu hình tối ưu sẵn có của Hermes
B4 – Jev Router	`typesafe/jev-router` làm main model, cost tier medium	Toàn bộ tool	Không có tín hiệu sandbox	Router thương mại, hộp đen
B5 – Rule-based	Luật theo độ dài câu lệnh và pha	Toàn bộ tool	Leo thang khi lỗi	Router đơn giản tự viết
B6 – Random	Ngẫu nhiên theo đúng tỷ lệ tầng của SCER-AH	Toàn bộ tool	Không	Kiểm tra router có thật sự học được gì
SCER-AH	Theo bước, Jev + hiệu chỉnh + entropy + chi phí chuyển	Lọc theo kỹ năng	Leo thang + AST gate	Đề xuất

B4 là baseline quan trọng nhất: nếu SCER-AH không thắng được Jev Router về chi phí ở cùng mức thành công, đóng góp sẽ yếu. B6 trả lời câu hỏi “giảm chi phí có phải chỉ do dùng nhiều mô hình rẻ hơn”.

6.2. Ablation – tắt từng thành phần

Biến thể	Tắt gì	Câu hỏi trả lời
−Skill	Bỏ s_skill khỏi trạng thái và không tiêm SKILL.md	Độ phủ kỹ năng đóng góp bao nhiêu?

Biến thể	Tắt gì	Câu hỏi trả lời
−Prune	Giữ toàn bộ tool khi chạy Fast	Cắt tỉa công cụ giảm lỗi gọi hàm bao nhiêu?
−AST	Bỏ AST gate và ép kiểu	Chốt chặn tiết kiệm bao nhiêu vòng sửa lỗi?
−Cascade	Không leo thang khi lỗi	Leo thang cứu bao nhiêu tác vụ?
−Entropy	Chỉ dùng ngưỡng τ_p	Entropy có giảm ca định tuyến sai không?
−SwitchCost	Bỏ luật chi phí chuyển	Bỏ qua cache làm tăng chi phí thật bao nhiêu?
Jev → LightGBM	Thay bộ ước lượng	Mô hình quyết định có sẵn so với tự huấn luyện

6.3. Quy trình chạy

1. **Pilot (20–30 tác vụ):** chạy song song Fast và Frontier ở từng bước để đo P1–P4 và gán nhãn "tầng tối thiểu đủ" cho ~300 bước.
2. **Hiệu chỉnh:** học α, β, T và chọn τ_p, τ_H trên tập pilot; không dùng lại tập này để báo cáo kết quả.
3. **Đánh giá chính:** chạy mọi cấu hình trên bộ benchmark, mỗi cấu hình lặp 3 lần để có trung bình và độ lệch chuẩn.
4. **Hai pha:** cold-start (kho kỹ năng rỗng) và warm-start (sau khi đã tích lũy kỹ năng từ một nửa tác vụ, đánh giá trên nửa còn lại).

7. Benchmark, metric và biểu đồ đánh giá

Đánh giá dùng bộ 100 tác vụ tự xây dựng, mô phỏng yêu cầu thật gửi qua Zalo, mỗi tác vụ có bộ kiểm tra tự động để không phải chấm tay.

7.1. Bộ benchmark

Nhóm	Số tác vụ	Ví dụ	Cách chấm tự động
L1 – Thống kê cơ bản	35	Doanh thu trung bình theo tháng, top 5 sản phẩm	So khớp số với đáp án tính sẵn (sai số ⩽ 1%)
L2 – Dữ liệu bẩn + biểu đồ	40	Xử lý ô trống, sai định dạng ngày, rồi vẽ biểu đồ đường	Kiểm tra số liệu trung gian + file ảnh tồn tại + loại biểu đồ đúng
L3 – Phối hợp đa công cụ	25	Gộp hai file, lọc, vẽ nhiều biểu đồ, viết tóm tắt tiếng Việt	Kiểm tra từng đầu ra; tóm tắt chấm bằng LLM-judge có rubric

Một nửa tác vụ trong mỗi nhóm là biến thể của nhau (cùng dạng, khác dữ liệu) để đo hiệu ứng tái sử dụng kỹ năng ở pha warm-start. Dữ liệu dùng bộ công khai hoặc tự sinh, không dùng dữ liệu cá nhân.

7.2. Metric

Nhóm	Metric	Đơn vị
Hiệu quả	Task Success Rate (TSR); Pass@1 (đúng ngay không cần sửa); Recovery Rate (thành công sau khi đã lỗi)	%
Chi phí	Tổng chi phí API trên 100 tác vụ, gồm cả phí gọi Jev và phần mất cache	USD
Độ trễ	Thời gian từ lúc nhận tin Zalo đến lúc gửi kết quả (trung bình, p95)	giây
Ngữ cảnh	Token đầu vào mỗi bước và mỗi tác vụ	token
Khung thực thi	Tỷ lệ lỗi schema/JSON, lỗi AST, lỗi runtime; số bước sửa lỗi trung bình	%, bước
Định tuyến	Offload Ratio (tỷ lệ bước giao cho Fast/Specialist); số lần chuyển tầng mỗi tác vụ	%, lần
Hiệu chỉnh	ROC-AUC, Brier score, Expected Calibration Error (ECE) của $\hat{p}$	–

Metric tổng hợp để so sánh: **chi phí trên mỗi tác vụ thành công** (USD / tác vụ đúng), tránh việc một cấu hình rẻ nhưng làm sai nhiều trông có vẻ tốt.

7.3. Biểu đồ đưa vào báo cáo và dashboard

1. **Cột ghép:** TSR, chi phí, độ trễ của B1–B6 và SCER-AH.
2. **Đường cong Pareto:** chi phí (trục x) và TSR (trục y) khi quét τ_p từ 0,1 đến 0,9; các baseline là các điểm đơn.
3. **Cột chồng theo pha:** tỷ lệ bước Fast / Specialist / Frontier ở pha lập kế hoạch, viết code, sửa lỗi, trả lời.
4. **Đường học cold → warm:** trục x là số tác vụ đã chạy, trục y là tỷ lệ gọi Frontier và độ trễ trung bình.
5. **Phân rã lỗi gọi hàm:** lỗi JSON, nhầm tool, lỗi runtime khi bật/tắt cắt tỉa và AST gate.
6. **Token ngữ cảnh theo bước:** bước 1 → 10, có và không có nén đầu ra công cụ.
7. **Reliability diagram + ROC:** chất lượng xác suất trước và sau hiệu chỉnh.
8. **Ablation:** mức sụt TSR và tăng chi phí khi tắt từng thành phần.

8. Kế hoạch triển khai và tiến độ

Phần cài đặt SCER-AH (tuần 5–8) là khối việc lớn nhất; benchmark và pilot phải xong trước đó vì chúng cung cấp nhãn để hiệu chỉnh router.

Đề tài hoàn thành trong 15 tuần, qua 6 mốc kiểm tra



lộ trình 15 tuần · 6 giai đoạn, 6 mốc

Lịch dựa trên một học kỳ 15 tuần; cần khớp lại với lịch báo cáo thực tế của học phần và giảng viên hướng dẫn. Nếu trễ, có thể cắt bớt tầng Specialist (chỉ giữ Fast và Frontier) mà không mất đóng góp chính.

Bố cục báo cáo Đồ án 2 đề xuất:

1. Chương 1 – Đặt vấn đề: đánh đổi chi phí – độ trễ – độ chính xác của tác tử nhiều bước trên nền tảng nhắn tin; số liệu pilot P1–P4.
2. Chương 2 – Cơ sở lý thuyết và khoảng trống: kiến trúc Hermes, RouteLLM, FrugalGPT, Jev Router; phát biểu khoảng trống.
3. Chương 3 – Phương pháp SCER-AH: vector trạng thái, router hai pha, hiệu chỉnh, entropy, chi phí chuyển, harness thích ứng, cascade, kiến trúc Zalo + dashboard.
4. Chương 4 – Thực nghiệm và đánh giá: benchmark, so sánh baseline, ablation, đường học, ảnh demo.

9. Rủi ro, hạn chế và điểm cần kiểm chứng

Rủi ro lớn nhất là chi phí mất prompt cache khi chuyển mô hình có thể ăn hết phần tiết kiệm; đề tài xử lý bằng luật chi phí chuyển và đo riêng khoản này.

Rủi ro	Ảnh hưởng	Cách giảm
Mất prompt cache khi đổi mô hình giữa bước	Chi phí thật cao hơn ước tính	Luật chi phí chuyển; ưu tiên đổi tầng sớm; ablation −SwitchCost
Jev là mô hình mới (ra mắt 15/09/2026), API Decisions đang ở bản alpha	Thay đổi định dạng, xác suất trôi giữa phiên bản	Ghim phiên bản `typesafe/jev-1.14`; có phương án LightGBM thay thế
Can thiệp vào vòng lặp chính của Hermes khó hơn dự kiến	Trễ tiến độ	Bắt đầu bằng proxy OpenAI-compatible đứng trước Hermes; chỉ sửa core khi cần đổi toolset
Bộ benchmark tự xây dựng có thể thiên lệch	Kết quả khó tổng quát	Công bố bộ tác vụ; tách tập pilot và tập đánh giá; báo cáo theo từng nhóm L1–L3
Chi phí API cho thực nghiệm	Vượt ngân sách sinh viên	Ước tính trước bằng pilot; B1 chạy 1 lần thay vì 3 nếu cần
Kết quả LLM dao động giữa các lần chạy	Kết luận không chắc chắn	Lặp 3 lần, báo trung bình ± độ lệch chuẩn

Hạn chế phạm vi: đề tài chỉ xét miền phân tích dữ liệu dạng bảng và mô hình đám mây; không huấn luyện lại LLM, không chạy mô hình cục bộ do giới hạn GPU.

Các điểm trong cuộc trao đổi với Gemini cần sửa khi viết báo cáo chính thức:

- Các tỷ lệ phần trăm (60–75%, 30–45%, giảm 50–70% chi phí) chưa có nguồn; chỉ dùng sau khi đo trong pilot.
- Gemini cho rằng Jev chỉ đọc câu hỏi đầu tiên. Thực tế Jev Router có thể giữ, đổi mô hình hoặc chỉnh mức suy luận qua các lượt trong luồng agentic. Khác biệt đúng là: Jev Router không thấy trạng thái nội bộ của Hermes và không đổi công cụ, kỹ năng.
- Luật kép xác suất + entropy chỉ có nghĩa với phân phối nhiều lớp (đã sửa ở mục 4.3).
- Tên mô hình cụ thể cần cập nhật theo bảng giá hiện hành.

10. Tài liệu tham khảo

- Nous Research. Configuring Models – Hermes Agent. Main model, 11 khe auxiliary, fallback, chuyển mô hình giữa phiên và prompt cache.
- Nous Research. Hermes Agent – GitHub.
- OpenRouter. Jev Router. Cơ chế chọn mô hình và mức suy luận, cost tier, metadata định tuyến.
- OpenRouter. Jev Documentation – TypeSafe Decision Model. Decisions API, các primitive Choice / Noul / Score, cookbook Jev-verified cascade.
- DigitalOcean. What is Jev? TypeSafe AI's System One model. Ngày ra mắt, độ trễ và giá.
- Ong, I. et al. RouteLLM: Learning to Route LLMs with Preference Data. ICLR 2025.
- Chen, L., Zaharia, M., Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. 2023.
- Cuộc trao đổi gốc với Gemini: Đề xuất đề tài Đồ án Hermes Agent.